

FlowBoard Frontend

Complete React Project Interview Preparation and Revision Notes

- Repository analyzed: C:/Users/abhip/TaskManagementPlatform-Frontend
- Framework: React 19 + Vite + React Router v7
- Architecture: Context API + service layer + API gateway backend
- Deliverable type: Implementation-specific interview PDF (not generic theory)
- Generated on: May 25, 2026

Area	Observation
Project scale	77 tracked source/config files in repository
Global state	AuthContext and BoardContext (no Redux)
Auth model	JWT token in localStorage, ProtectedRoute, OAuth callback route
Backend connectivity	Axios instance + microservice endpoints via API gateway
Code quality notes	Build passes, lint has issues and improvement opportunities

Table of Contents

Table of Contents	2
1. Project Snapshot	6
1.1 Tech Stack and Libraries	6
1.2 Top Level Folder Structure	6
2. Component Hierarchy and Rendering Layers	6
3. Routing Structure and Protected Navigation	7
3.1 Route Definitions (src/App.jsx)	7
3.2 Protected Route Behavior	8
4. Authentication and Token Architecture	8
4.1 Core Auth Files	8
4.2 Token Handling Flow	8
4.3 OAuth Login Flow	8
4.4 Auth-Specific Interview Points	9
5. State Management Flow (Context API)	9
5.1 AuthContext State and Actions	9
5.2 BoardContext State and Actions	9
5.3 Board Data Loading Chain	9
6. API Service Layer and Backend Communication	10
6.1 Axios Configuration	10
6.2 Service Files and Microservice Boundaries	10
7. React Concepts Used in THIS Project	10
JSX	10
Components	11
Props	11

State	11
useState	12
useEffect	12
useRef	12
useMemo	13
useCallback	13
Custom Hooks	13
Context API	14
Redux / Redux Toolkit	14
Routing	14
Protected Routes	15
Axios / API calls	15
Forms	15
Event Handling	15
Conditional Rendering	16
LocalStorage / sessionStorage	16
Authentication	16
Token Handling	17
Error Handling	17
Reusable Components	17
Performance Optimization	18
Lazy Loading	18
Environment Variables	18
Rendering Flow	19
Component Communication	19
8. Complete Project Flow (File-by-File Execution)	19
8.1 Application Startup Flow	19

8.2 Username/Password Login Flow	20
8.3 OAuth Login Flow	20
8.4 API Request Flow	20
8.5 Token Expiry and Unauthorized Flow	21
8.6 Logout Flow	21
9. Important Reusable Utilities, Patterns, and Dependencies	21
9.1 Reusable Components	21
9.2 Permission Utility Layer	21
9.3 Important Dependencies from package.json	21
10. Common Beginner Mistakes Found Here and Improvement Ideas	22
10.1 Observed Risks From Current Code	22
10.2 Practical Improvement Plan	22
11. How to Explain This Project in Interview	22
11.1 Two-Minute Explanation	23
11.2 Five-Minute Explanation	23
11.3 HR Round Version	23
11.4 Technical Round Version	23
12. Top 100 Interview Questions From This Project	24
Architecture and Folder Design	24
Routing and Navigation	24
Authentication and Security	25
State Management and Hooks	25
API and Backend Communication	26
UI, Forms, and Events	26
Performance, Quality, and Testing	27
Scenario and Viva Questions	27

13. Final Revision Sections	27
13.1 Quick Revision Sheet	27
13.2 Important Definitions Summary	28
13.3 Important Hooks Summary	28
13.4 Important Files Summary	28
13.5 One-Line Interview Answers	28
13.6 Most Important Things to Revise Before Interview	29

1. Project Snapshot

This project is a task/workspace management frontend with role-aware collaboration features. The implementation is service-driven and context-driven: pages/components trigger actions from context, context calls API service functions, service functions call axios, and backend responses update context state and UI.

1.1 Tech Stack and Libraries

- React 19 and react-dom 19 for UI rendering.
- Vite 8 for development server and production build.
- react-router-dom 7 for routing and route guards.
- axios for backend communication and request interceptors.
- @hello-pangea/dnd for board/list/card drag-and-drop.
- framer-motion for landing page animation effects.
- lucide-react for iconography.
- CSS Modules for component-level styling.

1.2 Top Level Folder Structure

Folder/File	Purpose in this project
src/main.jsx	React entry point with StrictMode and App mount.
src/App.jsx	All route definitions and layout composition.
src/context	AuthContext and BoardContext global state/actions.
src/api	All backend service wrappers and axios instance.
src/pages	Route-level screens (Auth, Dashboard, Workspace, Board, Admin).
src/components	Reusable UI, modals, board/list/card features.
src/utils/permissions.js	Centralized role checks and JWT claim helpers.
.env.example	VITE_API_BASE_URL backend gateway URL setting.
package.json	Project scripts and dependency manifest.

2. Component Hierarchy and Rendering Layers

Primary runtime hierarchy from actual implementation:

main.jsx

```

-> App
  -> AuthProvider
    -> BrowserRouter
      -> Routes
        Public:
          / -> RootRoute -> LandingPage
          /login -> LoginRoute -> AuthPage
          /oauth-success -> OAuthSuccess
        Protected:
          ProtectedRoute -> AppLayout
            -> Navbar
            -> Sidebar
            -> Outlet page
              /dashboard -> Dashboard
              /public-dashboard -> PublicDashboard
              /workspaces/:workspaceId -> WorkspacePage
              /boards/:boardId -> BoardPage -> Board
              /tasks -> MyTasks
              /settings -> Settings
        Admin-only:
          ProtectedRoute(requirePlatformAdmin=true) -> AppLayout -> AdminPage

```

Nested Board hierarchy in /boards/:boardId route:

```

Board
  -> List (multiple)
    -> Card (multiple)
  -> CardModal (opened for create/edit/view)
    -> AttachmentUpload
    -> Comment thread UI
    -> Checklist and Label management
  -> BoardSettingsModal
  -> ConfirmModal (delete actions)

```

3. Routing Structure and Protected Navigation

3.1 Route Definitions (src/App.jsx)

```

<Route path="/" element={<RootRoute />} />
<Route path="/login" element={<LoginRoute />} />
<Route path="/oauth-success" element={<OAuthSuccess />} />

<Route element={<ProtectedRoute />>
  <Route element={<AppLayout />>
    <Route path="/dashboard" element={<Dashboard />} />
    <Route path="/public-dashboard" element={<PublicDashboard />} />
    <Route path="/workspaces/:workspaceId" element={<WorkspacePage />} />
    <Route path="/boards/:boardId" element={<BoardPage />} />
    <Route path="/tasks" element={<MyTasks />} />
    <Route path="/settings" element={<Settings />} />
  </Route>
</Route>

  <Route element={<ProtectedRoute requirePlatformAdmin />>

```

```

    <Route element={<AppLayout />}>
      <Route path="/admin" element={<AdminPage />} />
    </Route>
  </Route>

```

3.2 Protected Route Behavior

- File: src/components/ProtectedRoute.jsx
- If loading is true -> render loading text.
- If user is null -> redirect to /login and preserve from location.
- If admin route and user is not platform admin -> redirect to /dashboard.
- Else -> render Outlet.

4. Authentication and Token Architecture

4.1 Core Auth Files

- src/context/AuthContext.jsx
- src/api/authService.js
- src/api/axiosInstance.js
- src/pages/AuthPage/AuthPage.jsx
- src/App.jsx (OAuthSuccess, RootRoute, LoginRoute)

4.2 Token Handling Flow

- 1) User logs in from AuthPage -> useAuth().login(username, password)
- 2) AuthContext.login -> authService.login
- 3) token saved in localStorage["token"]
- 4) profile fetched by username -> localStorage["user"]
- 5) axios interceptor reads token for every request:
Authorization: Bearer <token>
- 6) On 401 response:
localStorage token/user cleared
browser redirected to "/"

4.3 OAuth Login Flow

- AuthPage builds GOOGLE_OAUTH_URL using VITE_API_BASE_URL + /auth-service/oauth2/authorization/google.
- After backend OAuth success, frontend route /oauth-success reads token from query parameter.
- OAuthSuccess in src/App.jsx calls loginWithToken(token).

- loginWithToken decodes JWT payload, extracts username from sub claim, fetches profile, persists session.
- Route then redirects to /dashboard.

4.4 Auth-Specific Interview Points

- Session rehydration is done at AuthProvider mount using localStorage token + cached user.
- Role normalization maps ADMIN to PLATFORM_ADMIN in permissions utility.
- Auth page enforces portal separation: admin login tab blocks non-admin user, and user tab blocks admin account.
- Global 401 handling is centralized in axios interceptor, not repeated in each API call.

5. State Management Flow (Context API)

5.1 AuthContext State and Actions

- State: user, loading.
- Actions: login, loginWithToken, register, logout, updateUser.
- Derived values: role, isPlatformAdmin.
- Persistence: localStorage token and user object.

5.2 BoardContext State and Actions

- State includes workspaces, publicWorkspaces, boards, lists, cards, member maps, loading flags, and error.
- Action groups: workspace CRUD, board CRUD, list CRUD/reorder, card CRUD/move/assignee, member management.
- Normalization helpers convert backend IDs like workspaceId/boardId/cardId to string id for React state.
- Permission gates are enforced before mutating APIs (ensureWorkspaceAccess, ensureBoardAccess).
- On user change, fetchWorkspaces and fetchAssignedBoards run automatically in useEffect.

5.3 Board Data Loading Chain

```
Auth user available
-> BoardContext.useEffect fetchWorkspaces(userId)
-> setActiveWorkspaceId
-> Sidebar/WorkspacePage triggers fetchBoards(workspaceId)
-> BoardPage sets activeBoardId from route param
-> BoardContext.useEffect fetchLists(activeBoardId)
-> for each list -> fetchCards(list.id)
-> Board component consumes getBoardLists/getListCards derived selectors
```

6. API Service Layer and Backend Communication

6.1 Axios Configuration

```
const API_BASE_URL = import.meta.env.VITE_API_BASE_URL || "http://localhost:8765";
const axiosInstance = axios.create({ baseURL: API_BASE_URL });

request interceptor:
  attach Authorization Bearer token if localStorage token exists

response interceptor:
  if status 401 -> clear localStorage token/user and redirect "/"
```

6.2 Service Files and Microservice Boundaries

Service file	Gateway path family	Main usage locations
authService.js	/auth-service/auth/*	AuthPage, AuthContext, Board/Card comments author hydrate, Avatar
workspaceService.js	/workspace-service/workspaces/*	BoardContext, Sidebar, AdminPage
boardService.js	/board-service/boards/*	BoardContext, MyTasks, AdminPage
listService.js	/list-service/lists/*	BoardContext list lifecycle
cardService.js	/card-service/cards/*	BoardContext card lifecycle, CardModal, MyTasks
commentService.js	/comment-service/*	CardModal comments and attachments
labelService.js	/label-service/*	Card and CardModal labels/checklists
notificationService.js	/notification-service/*	Navbar, AdminPage broadcast

Environment configuration is minimal and explicit: VITE_API_BASE_URL appears in src/api/axiosInstance.js and src/pages/AuthPage/AuthPage.jsx.

7. React Concepts Used in THIS Project

Each block below includes implementation-specific references from this repository.

JSX

- **Definition:** Declarative syntax that returns UI elements.
- **Why it is used:** It maps component state directly to visual output.
- **Why this project uses it:** All pages/components are written in JSX files under src/pages and src/components.
- **Where it is used:** Every rendered view in App routes and feature components.

- **Flow explanation:** Context state changes cause JSX trees to re-render with latest board/workspace/card data.
- **Interview explanation:** Say: JSX is the view layer that reflects context/service state in this app.
- **Common mistakes:** Putting business logic-heavy expressions inline and making JSX too dense.
- **Important interview points:** Use JSX structure to show component boundaries and ownership in interviews.

Components

- **Definition:** Reusable UI units with isolated logic and props.
- **Why it is used:** Encapsulation improves maintainability and reuse.
- **Why this project uses it:** Board, List, Card, CardModal, Navbar, Sidebar, Avatar, ConfirmModal are modular units.
- **Where it is used:** src/components/* and src/pages/*.
- **Flow explanation:** Parent components pass IDs/callbacks; child components trigger context actions or local UI updates.
- **Interview explanation:** Explain how page components orchestrate context while reusable components stay focused.
- **Common mistakes:** Overloading one component with too many responsibilities.
- **Important interview points:** Mention separation between route screens and shared UI widgets.

Props

- **Definition:** Inputs passed from parent component to child component.
- **Why it is used:** To pass data and actions down tree without global state for everything.
- **Why this project uses it:** Board passes list and handlers into List; List passes card and edit handler into Card.
- **Where it is used:** Board.jsx, List.jsx, Card.jsx, CardModal.jsx, ConfirmModal.jsx.
- **Flow explanation:** Props carry IDs, readOnly flags, callbacks like onEdit/onClose/onConfirm.
- **Interview explanation:** Highlight prop-driven composition in board/list/card stack.
- **Common mistakes:** Prop drilling too deep without context boundaries.
- **Important interview points:** Show where props are better than context for local relationships.

State

- **Definition:** Mutable data stored inside components or contexts.
- **Why it is used:** To drive reactivity in UI.
- **Why this project uses it:** Local UI state for forms/modals; global state in AuthContext and BoardContext.

- **Where it is used:** useState across pages/components plus context state maps.
- **Flow explanation:** API response updates state, state updates render.
- **Interview explanation:** Differentiate global domain state vs local interaction state.
- **Common mistakes:** Mixing transient UI state into global context unnecessarily.
- **Important interview points:** Interviewers like clear state ownership explanation.

useState

- **Definition:** Hook for local state in function components.
- **Why it is used:** Needed for forms, toggles, modal visibility, progress, filters.
- **Why this project uses it:** Used heavily for login form mode, board modal flags, sidebar dropdowns, my-task filters.
- **Where it is used:** AuthPage.jsx, Board.jsx, Sidebar.jsx, CardModal.jsx, Settings.jsx, others.
- **Flow explanation:** User action -> setState -> re-render with updated UI.
- **Interview explanation:** Give a concrete example: mode switch in AuthPage and form reset.
- **Common mistakes:** Forgetting to reset related fields when changing mode.
- **Important interview points:** State naming clarity matters in complex components.

useEffect

- **Definition:** Hook for side effects such as API calls and subscriptions.
- **Why it is used:** Needed for startup fetches and synchronization with route/context changes.
- **Why this project uses it:** AuthContext hydrates user on mount; BoardContext fetches data on user/board changes.
- **Where it is used:** AuthContext.jsx, BoardContext.jsx, Navbar.jsx, BoardPage.jsx, MyTasks.jsx.
- **Flow explanation:** Dependency change -> effect runs -> async service call -> set state.
- **Interview explanation:** Explain dependencies and why certain effects intentionally run on route change.
- **Common mistakes:** Missing dependency arrays or stale closures causing outdated data.
- **Important interview points:** Mention cleanup patterns and request race considerations.

useRef

- **Definition:** Mutable container that does not trigger render.
- **Why it is used:** Useful for DOM anchors and stable mutable references.
- **Why this project uses it:** Used for click-outside behavior, debounce timers, and board list reference in BoardPage.
- **Where it is used:** Board.jsx, Card.jsx, CardModal.jsx, Sidebar.jsx, Navbar.jsx, BoardPage.jsx.

- **Flow explanation:** Ref stores element or timer -> event handlers read/write without re-render.
- **Interview explanation:** Show why BoardPage uses boardsRef to avoid effect reruns on every boards update.
- **Common mistakes:** Using ref when state should be used for rendering data.
- **Important interview points:** Differentiate useRef vs useState clearly.

useMemo

- **Definition:** Memoizes derived values.
- **Why it is used:** Avoids recomputing expensive or repeated derivations.
- **Why this project uses it:** Used for sorted workspace lists, derived role/summary values, filtered tasks, nav links.
- **Where it is used:** Dashboard.jsx, Board.jsx, Sidebar.jsx, PublicDashboard.jsx, AdminPage.jsx.
- **Flow explanation:** Inputs change -> recompute memoized result -> render uses stable derived data.
- **Interview explanation:** Describe one concrete memo: boardSummary in Board.jsx.
- **Common mistakes:** Memoizing trivial values with no measurable gain.
- **Important interview points:** Use for derived collections used multiple times in render.

useCallback

- **Definition:** Memoizes function identity.
- **Why it is used:** Useful when callbacks are dependencies or passed down often.
- **Why this project uses it:** BoardContext exposes many stable action functions with useCallback.
- **Where it is used:** AuthContext.jsx and BoardContext.jsx.
- **Flow explanation:** Stable callback references reduce unnecessary effect triggers in consumers.
- **Interview explanation:** Point out how context action functions are wrapped and exported.
- **Common mistakes:** Wrapping every function blindly without benefit.
- **Important interview points:** useCallback is valuable in context providers to avoid value churn.

Custom Hooks

- **Definition:** Project-specific hooks built from existing hooks.
- **Why it is used:** Encapsulates context access and guard logic.
- **Why this project uses it:** useAuth and useBoard wrap useContext and enforce provider usage.
- **Where it is used:** AuthContext.jsx and BoardContext.jsx export these hooks.
- **Flow explanation:** Consumer imports hook -> gets state/actions from provider value.
- **Interview explanation:** Mention thrown error when hook is used outside provider.

- **Common mistakes:** Forgetting provider wrappers around route tree.
- **Important interview points:** Custom hooks improve developer ergonomics and safety.

Context API

- **Definition:** React mechanism for sharing global state without prop drilling.
- **Why it is used:** Ideal for cross-cutting app data.
- **Why this project uses it:** AuthContext for user/session and BoardContext for all workspace-board-list-card operations.
- **Where it is used:** src/context/AuthContext.jsx and src/context/BoardContext.jsx.
- **Flow explanation:** Provider wraps app layout, consumers pull state/actions via hooks.
- **Interview explanation:** Explain this as project's state backbone replacing Redux.
- **Common mistakes:** Huge monolithic context can become heavy if not segmented.
- **Important interview points:** Discuss tradeoff: simple architecture but large provider surface.

Redux / Redux Toolkit

- **Definition:** External global state library with reducers and predictable events.
- **Why it is used:** Common in larger apps with strict state/event architecture.
- **Why this project uses it:** Not used in this project. Context API is the chosen approach.
- **Where it is used:** No Redux store/slice files exist in repository.
- **Flow explanation:** State mutations happen via context action functions and setState.
- **Interview explanation:** Say: project intentionally uses Context for simpler setup.
- **Common mistakes:** Claiming Redux knowledge without mapping to actual code.
- **Important interview points:** Be explicit that no Redux middleware/devtools pipeline is present.

Routing

- **Definition:** URL-based view selection and navigation.
- **Why it is used:** Required for multi-screen product behavior.
- **Why this project uses it:** App routes include public pages, protected app pages, and admin-only pages.
- **Where it is used:** src/App.jsx with BrowserRouter, Routes, Route, Navigate, Outlet.
- **Flow explanation:** Route match -> component mount -> effects fetch relevant data.
- **Interview explanation:** Explain nested routes: ProtectedRoute + AppLayout + Outlet.
- **Common mistakes:** Hardcoding redirect logic in many places instead of central routes.
- **Important interview points:** Show route guards and fallback route behavior.

Protected Routes

- **Definition:** Guarded routes that require auth or role.
- **Why it is used:** Prevents unauthorized page access.
- **Why this project uses it:** ProtectedRoute checks loading, user existence, and platform admin requirement.
- **Where it is used:** src/components/ProtectedRoute.jsx and usage in src/App.jsx.
- **Flow explanation:** Unauthorized user redirected to login with original location state.
- **Interview explanation:** Mention admin route protection with requirePlatformAdmin prop.
- **Common mistakes:** Only protecting UI buttons but not routes.
- **Important interview points:** Route-level guard is first line before screen mount.

Axios / API calls

- **Definition:** HTTP client abstraction for backend communication.
- **Why it is used:** Centralizes headers, interceptors, and error behavior.
- **Why this project uses it:** All service files call shared axios instance; token handled automatically.
- **Where it is used:** src/api/axiosInstance.js and src/api/*.js.
- **Flow explanation:** Component/context action -> service function -> axios request -> response data -> state update.
- **Interview explanation:** Discuss gateway-based endpoint prefixes and modular services.
- **Common mistakes:** Calling backend directly from JSX instead of service layer.
- **Important interview points:** Service layer keeps UI code cleaner and testable.

Forms

- **Definition:** Controlled input patterns for user data entry.
- **Why it is used:** Needed for login, create/edit forms, settings, invites, comments.
- **Why this project uses it:** State-backed forms are used across AuthPage, Dashboard modal, WorkspacePage, CardModal, Settings.
- **Where it is used:** Multiple page/component files with input/select/textarea and submit handlers.
- **Flow explanation:** Input change updates state, submit triggers async action and loading/error feedback.
- **Interview explanation:** Use one concrete example: AuthPage mode-specific form behavior.
- **Common mistakes:** Not resetting form state on mode/context switch.
- **Important interview points:** Controlled components simplify validation and UX messaging.

Event Handling

- **Definition:** Responding to user actions (click, submit, keydown, blur, drag).
- **Why it is used:** Essential for interactive workflow tools.
- **Why this project uses it:** Drag/drop handlers, modal close handlers, key events for quick add/edit are used widely.
- **Where it is used:** Board.jsx, List.jsx, CardModal.jsx, Sidebar.jsx, AuthPage.jsx.
- **Flow explanation:** User event triggers function -> possibly async context action -> UI update.
- **Interview explanation:** Mention keyboard support examples (Enter/Escape in list/card forms).
- **Common mistakes:** Not stopping propagation in overlays leading to accidental closes.
- **Important interview points:** Event strategy improves productivity in board UI.

Conditional Rendering

- **Definition:** Rendering UI fragments based on conditions.
- **Why it is used:** To handle auth state, permissions, loading, empty states.
- **Why this project uses it:** Guest/member/admin UI differences are heavily condition-based.
- **Where it is used:** ProtectedRoute, Board, WorkspacePage, Sidebar, MyTasks, AuthPage.
- **Flow explanation:** State/role flags decide visibility of actions, modals, and sections.
- **Interview explanation:** Explain how permissions utility drives condition checks.
- **Common mistakes:** Scattered role logic without central utility.
- **Important interview points:** Centralized permission functions improve consistency.

LocalStorage / sessionStorage

- **Definition:** Browser persistence mechanisms.
- **Why it is used:** To retain session between page refreshes.
- **Why this project uses it:** localStorage is used for token and user cache; sessionStorage is not used.
- **Where it is used:** AuthContext.jsx and axiosInstance.js contain localStorage logic.
- **Flow explanation:** On app init AuthContext reads localStorage; on logout/401 storage is cleared.
- **Interview explanation:** State clearly: sessionStorage is absent in this implementation.
- **Common mistakes:** Storing sensitive data without expiration strategy.
- **Important interview points:** Know where persistence is read and invalidated.

Authentication

- **Definition:** Identity verification process before protected access.
- **Why it is used:** Needed to secure workspace and board data.
- **Why this project uses it:** Supports username/password login/register and Google OAuth callback flow.

- **Where it is used:** AuthPage.jsx, AuthContext.jsx, App.jsx OAuthSuccess, authService.js.
- **Flow explanation:** Login -> token + profile -> context user -> protected pages unlocked.
- **Interview explanation:** Mention portal gating between admin and non-admin logins.
- **Common mistakes:** Trusting frontend-only role checks without backend authorization.
- **Important interview points:** Frontend checks improve UX but backend remains source of truth.

Token Handling

- **Definition:** Creating, storing, attaching, and invalidating JWT tokens.
- **Why it is used:** Required for authenticated API requests.
- **Why this project uses it:** Token saved after login, attached in interceptor, decoded for role/sub in context.
- **Where it is used:** AuthContext buildSessionUser + permissions.decodeJwtClaims + axios interceptor.
- **Flow explanation:** token -> decode claims -> fetch profile -> normalize role -> persisted session.
- **Interview explanation:** Be ready to explain why both token claims and fetched profile are used.
- **Common mistakes:** Only decoding token and skipping profile refresh can cause stale user data.
- **Important interview points:** This project combines token claim read with profile fetch for better data quality.

Error Handling

- **Definition:** Capturing and surfacing failures in async operations.
- **Why it is used:** Important for user trust and recoverability.
- **Why this project uses it:** Most async actions catch request errors and set message state; axios handles global 401.
- **Where it is used:** AuthPage, Dashboard, WorkspacePage, CardModal, Settings, BoardContext.
- **Flow explanation:** API failure -> catch -> setError or fallback value -> UI feedback.
- **Interview explanation:** Discuss layered handling: local component errors plus global unauthorized behavior.
- **Common mistakes:** Silent catches hide actionable failures.
- **Important interview points:** Balance user-friendly errors with logging for diagnostics.

Reusable Components

- **Definition:** Shared components that reduce duplication.
- **Why it is used:** Consistency and maintainability.
- **Why this project uses it:** Avatar, ConfirmModal, SkeletonLoader, AttachmentUpload, ColorWheelPicker are reused.
- **Where it is used:** src/components/Avatar, ConfirmModal, SkeletonLoader, etc.

- **Flow explanation:** Feature screens compose these blocks instead of re-implementing UI each time.
- **Interview explanation:** Show how AvatarStack and ConfirmModal are used across pages.
- **Common mistakes:** Building screen-specific duplicates with slight style differences.
- **Important interview points:** Reusable design system components speed feature delivery.

Performance Optimization

- **Definition:** Reducing unnecessary re-renders and expensive work.
- **Why it is used:** Needed because board UI can be data-heavy.
- **Why this project uses it:** useMemo/useCallback used in contexts and screens; optimistic deleteCard improves perceived speed.
- **Where it is used:** BoardContext, Dashboard, Sidebar, Board, MyTasks.
- **Flow explanation:** Derived data and stable callbacks minimize churn in nested components.
- **Interview explanation:** Mention build warning: large chunk suggests code-splitting opportunity.
- **Common mistakes:** Ignoring bundle size and rerender hotspots in board views.
- **Important interview points:** Performance talk should include render, network, and bundle concerns.

Lazy Loading

- **Definition:** Dynamic import of route/component chunks.
- **Why it is used:** Improves initial bundle load in large apps.
- **Why this project uses it:** Not implemented currently; routes are eagerly imported in App.jsx.
- **Where it is used:** No React.lazy or Suspense usage found.
- **Flow explanation:** Entire route component code ships in main bundle.
- **Interview explanation:** Use this as improvement: lazy-load admin/landing/board-heavy pages.
- **Common mistakes:** Assuming Vite auto-splits everything without dynamic imports.
- **Important interview points:** Observed build output has >500k JS chunk warning.

Environment Variables

- **Definition:** Build-time configuration values prefixed with VITE_.
- **Why it is used:** Separates environment-specific URLs from code.
- **Why this project uses it:** VITE_API_BASE_URL drives API gateway URL and OAuth URL construction.
- **Where it is used:** src/api/axiosInstance.js and src/pages/AuthPage/AuthPage.jsx.
- **Flow explanation:** If env value missing, fallback defaults to http://localhost:8765.
- **Interview explanation:** Explain how .env.example documents expected variable.
- **Common mistakes:** Forgetting VITE_ prefix in Vite projects.

- **Important interview points:** Environment config should be validated during deployment.

Rendering Flow

- **Definition:** How state, context, and route changes produce UI updates.
- **Why it is used:** Critical for debugging and explaining behavior.
- **Why this project uses it:** App mount -> auth hydrate -> route guard -> layout -> context-driven data fetch -> child render.
- **Where it is used:** main.jsx, App.jsx, AuthContext.jsx, BoardContext.jsx, page and component files.
- **Flow explanation:** Any context setState cascades through subscribed components.
- **Interview explanation:** Give timeline from login to board cards rendered.
- **Common mistakes:** Triggering broad context updates for tiny UI-only changes.
- **Important interview points:** Knowing render flow helps answer stale data and race condition questions.

Component Communication

- **Definition:** Data flow between parent-child and via context.
- **Why it is used:** Needed to coordinate large feature trees.
- **Why this project uses it:** Parent-child props handle local interactions; context handles shared domain actions/data.
- **Where it is used:** Board -> List -> Card chain plus CardModal callbacks; global actions via useBoard.
- **Flow explanation:** Child action calls context function, context updates store, all consumers re-render.
- **Interview explanation:** Explain where prop callbacks end and context begins.
- **Common mistakes:** Mixing both patterns inconsistently causing hard-to-trace updates.
- **Important interview points:** Clear communication boundaries are key interview signal.

8. Complete Project Flow (File-by-File Execution)

8.1 Application Startup Flow

- 1 src/main.jsx mounts App inside StrictMode.
- 2 src/App.jsx wraps routes with AuthProvider then BrowserRouter.
- 3 AuthProvider in src/context/AuthContext.jsx runs mount effect.
- 4 AuthProvider reads localStorage token and user cache.
- 5 If cache exists, buildSessionUser normalizes role and sets user state.
- 6 AuthProvider refreshes profile using authService.getUserByUsername.

- 7 Routes evaluate RootRoute or LoginRoute based on user/loading.
- 8 After authenticated route enters AppLayout, BoardProvider initializes.
- 9 BoardProvider runs user-based effect to fetch workspaces and assigned boards.
- 10 Sidebar/WorkspacePage select active workspace and trigger fetchBoards.
- 11 BoardPage sets activeBoardId from URL and BoardContext fetches lists/cards.
- 12 Board, List, and Card components render live state from context selectors.

8.2 Username/Password Login Flow

- 1 User submits AuthPage login form in src/pages/AuthPage/AuthPage.jsx.
- 2 handleSubmit calls useAuth().login(username, password).
- 3 AuthContext.login calls authService.login to get token and auth user payload.
- 4 AuthContext stores token in localStorage.
- 5 AuthContext fetches profile via authService.getUserByUsername.
- 6 Session user normalized and stored in localStorage user.
- 7 ProtectedRoute now sees user and grants access.
- 8 Route redirects to /dashboard or /admin based on role.

8.3 OAuth Login Flow

- 1 AuthPage redirects browser to backend OAuth URL.
- 2 Backend returns to /oauth-success?token=...
- 3 OAuthSuccess in App.jsx extracts token and calls loginWithToken.
- 4 AuthContext.loginWithToken decodes token, extracts sub username, fetches profile, persists user.
- 5 User is redirected to /dashboard.

8.4 API Request Flow

- 1 Component or page triggers context action (example: addCard from List).
- 2 Context action in BoardContext validates permission helpers.
- 3 BoardContext calls service function (cardService.createCard).
- 4 Service function calls shared axiosInstance with gateway route.
- 5 Request interceptor adds Authorization header if token exists.
- 6 Backend responds with card payload.
- 7 BoardContext normalizes response and updates cards state.

- 8 React re-renders List/Card UI with new card data.

8.5 Token Expiry and Unauthorized Flow

- 1 Any API call returning 401 triggers axios response interceptor.
- 2 Interceptor removes localStorage token and user cache.
- 3 Window location is changed to root route /.
- 4 RootRoute sends unauthenticated user to LandingPage.

8.6 Logout Flow

- 1 User clicks Sign Out in Navbar user menu.
- 2 logout from AuthContext clears localStorage token and user and sets user state null.
- 3 ProtectedRoute sees no user and redirects to /login.

9. Important Reusable Utilities, Patterns, and Dependencies

9.1 Reusable Components

- Avatar and AvatarStack for identity display across dashboard, board members, comments, and settings.
- ConfirmModal for destructive action confirmation (delete workspace/board/member).
- SkeletonLoader variants for loading placeholders in dashboard/sidebar/members.
- AttachmentUpload for comment attachment upload and preview.
- ColorWheelPicker for board background and card cover color selection.

9.2 Permission Utility Layer

- File: src/utis/permissions.js centralizes role derivation and capability checks.
- Functions include canManageWorkspace, canCreateBoard, canManageBoard, canCollaborateOnBoard, isGuestOnBoard, and canEditComment.
- BoardContext and UI components both use same utility to keep behavior consistent.

9.3 Important Dependencies from package.json

- react, react-dom: core UI runtime.

- react-router-dom: route tree, navigation, protected routes.
- axios: service layer and auth interceptors.
- @hello-pangea/dnd: drag-and-drop list/card movement.
- framer-motion: landing page animations and reveal effects.
- lucide-react: icon consistency across all screens.
- vite: development/build toolchain.

10. Common Beginner Mistakes Found Here and Improvement Ideas

10.1 Observed Risks From Current Code

- src/components/Settings/Settings.jsx uses `in` in JSX but `Loader` is not imported. This is a runtime bug path when password update is loading.
- Build output warns main JS chunk is large (around 686 KB before gzip).
- Unused artifacts exist: Timeline component, Logo component, WorkspaceSetup page, `initialData` mocks are currently not routed/used.
- Navbar search box is UI-only and not wired to any filtering/search service.
- Several API catches are silent and can hide operational failures (especially in CardModal and member fetch paths).
- Large BoardContext combines many concerns; future split by domain slice could improve maintainability.
- Lint run reports multiple unused vars and hook-related warnings/errors.

10.2 Practical Improvement Plan

- 1 Add route-level code splitting with `React.lazy` for AdminPage, BoardPage, and LandingPage.
- 2 Split BoardContext into focused contexts/hooks (workspace, board, card, membership).
- 3 Wire navbar search to board/card search and debounced service endpoint.
- 4 Standardize error handling helpers so silent catches become user-safe plus logged.
- 5 Introduce React Query or a caching layer for better request dedupe and stale-data handling.
- 6 Add unit tests for permission utilities and integration tests for auth/protected flows.

11. How to Explain This Project in Interview

11.1 Two-Minute Explanation

I built a React and Vite based task management frontend called FlowBoard. The app is structured around workspaces, boards, lists, and cards. For state management, I used Context API with two providers: AuthContext for user/session management and BoardContext for domain data and actions. Authentication is JWT based with localStorage persistence and an axios interceptor that attaches tokens and handles unauthorized responses globally. Routing uses React Router with protected routes and admin-only route checks. The board experience supports drag-and-drop using @hello-pangea/dnd, card modals with comments, attachments, labels, and checklists, and role-based permissions through a centralized permissions utility. Backend integration is modularized through service files for each microservice domain routed via an API gateway URL.

11.2 Five-Minute Explanation

- 1 Start with architecture: App routes, AuthProvider, BoardProvider, and shell layout.
- 2 Explain authentication lifecycle: login/register/oauth-success, token storage, profile fetch, protected route checks.
- 3 Explain domain flow: fetch workspaces -> fetch boards -> fetch lists -> fetch cards -> render board hierarchy.
- 4 Explain permission design: centralized utilities control both backend action guards and UI capability rendering.
- 5 Explain collaboration features: board members, workspace members, card comments, attachments, checklist and labels.
- 6 Explain service layer: each microservice has dedicated file and shared axios instance with interceptors.
- 7 Close with improvements: code-splitting, context modularization, better test coverage, and stronger error telemetry.

11.3 HR Round Version

I built a collaborative productivity platform frontend where teams can manage work in real time through workspaces and boards. I focused on user experience, clean component design, secure authentication, and clear separation between UI and backend integration. I also implemented role-based access so the same product safely supports normal users and platform admins.

11.4 Technical Round Version

The frontend is React 19 with Vite. Routing uses React Router v7 with nested protected routes and admin guards. Global state is handled with Context API: AuthContext for session and BoardContext for workspace-board-list-card data/actions. All network requests go through axios services with a gateway base URL and auth interceptors. IDs from backend payloads are normalized into consistent string IDs.

Permission checks are centralized in `utils/permissions` and enforced before mutating actions. Drag-and-drop is implemented using `hello-pangea/dnd`. The card modal handles comments, labels, checklists, and attachments with dedicated services.

12. Top 100 Interview Questions From This Project

Architecture and Folder Design

- 1 Explain the purpose of `src/App.jsx` in this project.
- 2 Why does this project use both `AuthProvider` and `BoardProvider`?
- 3 What responsibilities belong to pages vs components in this repository?
- 4 How is microservice backend complexity hidden from UI files?
- 5 Why is permissions logic kept in `src/utils/permissions.js`?
- 6 How would you onboard a new developer using this structure?
- 7 What are tradeoffs of a large `BoardContext` provider?
- 8 Which files are best candidates for module split next?
- 9 Why are API files separated by domain instead of one client file?
- 10 Where does cross-cutting auth logic live and why?
- 11 How would you add analytics tracking with minimal coupling?
- 12 How does the app shell support consistent navigation UX?
- 13 Which components are reusable across many pages and why?
- 14 What parts of this project are currently dead code?
- 15 How would you document ownership boundaries for future scaling?

Routing and Navigation

- 1 How does `RootRoute` decide between landing and authenticated redirects?
- 2 How does `LoginRoute` prevent logged-in users from visiting login page?
- 3 What is the purpose of `state.from` in `ProtectedRoute` redirect?
- 4 How is admin-only navigation enforced?
- 5 Why is `AppLayout` nested inside protected routes?
- 6 How does `BoardPage` sync URL `boardId` with `activeBoardId` context state?
- 7 What happens if a user manually opens `/admin` without admin role?
- 8 Where would you add a new protected route for reports?

- 9 How does fallback route handling work for unknown URLs?
- 10 Would you lazy-load route components and why?

Authentication and Security

- 1 Walk through login flow from AuthPage submit to dashboard render.
- 2 How does JWT token get attached to outgoing API calls?
- 3 Where is token stored and what are risks of that approach?
- 4 How does app restore session after refresh?
- 5 How does OAuth success callback work in this project?
- 6 Why does AuthContext still fetch profile after decoding token?
- 7 How is platform admin role normalized from backend values?
- 8 What happens on API 401 response globally?
- 9 How does AuthPage block admin login from normal user tab?
- 10 How does logout affect route access immediately?
- 11 How would you add token refresh support?
- 12 How would you reduce XSS risk around localStorage token usage?
- 13 How would you support session timeout warning UX?
- 14 What backend checks must still exist even with ProtectedRoute?
- 15 How would you test auth regression safely?

State Management and Hooks

- 1 Why choose Context API instead of Redux in this project?
- 2 What data belongs in AuthContext vs BoardContext?
- 3 Which BoardContext state fields are derived vs source-of-truth?
- 4 How are backend IDs normalized and why is that important?
- 5 How does fetchWorkspaces set active workspace?
- 6 How do lists and cards stay in sync with activeBoardId?
- 7 Explain optimistic delete behavior in deleteCard.
- 8 Where is useMemo used effectively in this codebase?
- 9 Where is useRef used for non-render mutable state?
- 10 How do custom hooks useAuth/useBoard improve safety?
- 11 How would you avoid stale closures in large context actions?

- 12 What are the risks of too many state fields in one provider?
- 13 How would you split context while keeping API ergonomics good?
- 14 How do loading flags impact user experience in this app?
- 15 How would you add offline cache for board data?

API and Backend Communication

- 1 What is the role of axiosInstance.js in this architecture?
- 2 How are auth, workspace, board, list, card services separated?
- 3 Explain request path format used with API gateway.
- 4 Which component triggers getCardsByAssignee and why?
- 5 How are comments and replies loaded in CardModal?
- 6 How are file uploads handled for avatars and attachments?
- 7 How does BoardContext ensure board access before mutation?
- 8 How does reorderBoardLists persist drag-and-drop ordering?
- 9 How is card move persisted after local state change?
- 10 Where are notification APIs consumed in UI?
- 11 How would you centralize API error normalization?
- 12 How would you add retry/backoff for flaky endpoints?
- 13 How would you instrument API latency per feature?
- 14 How would you protect against duplicate submit requests?
- 15 How would you migrate this service layer to TypeScript safely?

UI, Forms, and Events

- 1 How does AuthPage manage multiple modes in one component?
- 2 How does Board component restrict UI actions for guests?
- 3 How does List component handle inline add/edit interactions?
- 4 How does CardMenu avoid clipping inside overflow parents?
- 5 How does CardModal manage many feature states without external store?
- 6 How are keyboard shortcuts handled for fast add/edit patterns?
- 7 How does Sidebar switch active workspace and trigger navigation?
- 8 How is form validation handled in settings password flow?
- 9 How are empty and loading states represented in dashboard/tasks?

10 How would you improve accessibility across modal components?

Performance, Quality, and Testing

- 1 What does the large production chunk warning indicate?
- 2 Which routes would you lazy-load first and why?
- 3 Where can memoization be removed or improved?
- 4 What runtime bug exists in Settings component today?
- 5 Which lint findings matter most for reliability?
- 6 How would you test drag-and-drop behavior deterministically?
- 7 How would you test permission matrix quickly?
- 8 How would you prevent over-fetching in MyTasks refresh flow?
- 9 How would you monitor re-render hotspots in Board/Card tree?
- 10 What CI checks would you enforce before merge?

Scenario and Viva Questions

- 1 A user can open board URL but sees no cards. How do you debug from route to API?
- 2 Assigned board appears without workspace name. Which states/services are involved?
- 3 Card deletion disappears then comes back. Explain possible optimistic update rollback path.
- 4 Admin route intermittently redirects to dashboard. What auth role checks will you inspect?
- 5 Notifications badge count is wrong after read-all. Where will you inspect state mutations?
- 6 OAuth login returns token but user stays on processing state. What functions and failure points are involved?
- 7 Board drag-and-drop order looks right until refresh. Which persistence call may be failing?
- 8 Attachment upload succeeds but preview fails for image. Which fields and components do you inspect?
- 9 A member should not edit board but can. Which permission utilities and UI conditions are relevant?
- 10 How would you present this architecture to a senior panel in 3 minutes?

Total question count in this section: 100

13. Final Revision Sections

13.1 Quick Revision Sheet

- Global auth/session flows are handled in AuthContext plus axios interceptors.
- Domain data flow starts in BoardContext and fans out to pages/components.
- Permissions are centralized in utils/permissions.js and reused in both context and UI.
- Board hierarchy rendering is Board -> List -> Card with CardModal as detail editor.
- Microservice API communication is encapsulated in src/api service files.

13.2 Important Definitions Summary

- Workspace: top-level collaboration container.
- Board: task canvas inside a workspace.
- List: Kanban column inside a board.
- Card: task item with status, priority, assignee, dates, labels, checklist.
- Platform Admin: special role with global access and admin dashboard route.

13.3 Important Hooks Summary

- useAuth: session and auth operations.
- useBoard: workspace-board-list-card state and actions.
- useEffect: data loading and synchronization across auth, board, and page lifecycles.
- useMemo/useCallback: performance and stable dependency support.
- useRef: click-outside handling, timers, and stable mutable references.

13.4 Important Files Summary

- src/App.jsx -> full route map and auth/public entry logic.
- src/context/AuthContext.jsx -> login/register/logout/session hydrate logic.
- src/context/BoardContext.jsx -> core business state and API actions.
- src/api/axiosInstance.js -> token attach and 401 fallback.
- src/components/Board/Board.jsx -> board UI, drag-drop, member management entry.
- src/components/CardModal/CardModal.jsx -> richest task-detail workflow.
- src/utils/permissions.js -> role and capability checks.

13.5 One-Line Interview Answers

- 1 State strategy: Context API with AuthContext and BoardContext, no Redux.
- 2 Auth strategy: JWT in localStorage with axios request/response interceptors.

- 3 Routing strategy: nested protected routes with admin-only guard.
- 4 Data strategy: service files per microservice domain through API gateway.
- 5 Permission strategy: centralized utility functions reused across UI and actions.
- 6 Board UX strategy: drag-drop lists/cards with optimistic updates for responsiveness.
- 7 Collaboration strategy: members, comments, attachments, labels, and checklists at card level.
- 8 Scalability concern: BoardContext is powerful but large; future split is recommended.
- 9 Performance concern: large bundle size suggests route-level lazy loading.
- 10 Quality concern: lint findings include one runtime bug and multiple cleanup opportunities.

13.6 Most Important Things to Revise Before Interview

- 1 Be able to walk login flow end-to-end with exact file names.
- 2 Memorize route guard behavior and how admin route differs.
- 3 Understand BoardContext fetch chain and normalization helpers.
- 4 Know one example each of useMemo, useCallback, and useRef in this codebase.
- 5 Know where localStorage is read/written/cleared.
- 6 Know service-layer pattern and gateway endpoint prefixes.
- 7 Prepare one improvement story: lazy loading + context modularization + tests.